

[ETS] Coding demo

Author(s): Ilyasa' Sirojuddin SYafiq Ashari
 Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
 Class name: Sistem Robot Otonom
 YouTube video link: <https://youtu.be/-dHkTUoRkJ4>
 GitHub link: <https://github.com/syafiqilya/Sistem-Robot-Otonom>

1 Pendahuluan

Sistem robotika otonom menuntut kemampuan navigasi dan interaksi objek secara presisi dalam lingkungan yang dinamis. Pada simulasi ini, sistem yang dibangun memodelkan interaksi multi-robot pada skenario permainan sepak bola sederhana menggunakan simulator CoppeliaSim. Pengendalian setiap agen didasarkan pada pemodelan kinematika *differential drive* yang dieksekusi secara *real-time* melalui antarmuka Python ZeroMQ Remote API.

Tujuan utama dari eksperimen dan laporan ini adalah menganalisis kinerja algoritma pelacakan target (*target tracking*) dan manuver patroli otonom. Analisis secara khusus difokuskan pada dua entitas utama:

1. **Robot Striker (Robot_Pemain):** Mengimplementasikan pengontrol kecepatan proporsional untuk menghitung jarak dan sudut orientasi terhadap bola (**Bola_Merah**), kemudian menggiringnya menuju titik target absolut yang berjarak 1,0 meter di dalam area gawang (**Gwang_Kuning**).
2. **Robot Goalkeeper (Robot_Lawan_01):** Mengimplementasikan mekanisme patroli spasial statis dengan bergerak secara linear bolak-balik (osilasi sumbu-*y*) di depan garis gawang sebagai rintangan dinamis.

Eksperimen ini dirancang untuk mengobservasi bagaimana penyesuaian parameter kontrol—khususnya batas kecepatan linear maksimum dan nilai penguatan sudut (*angular turn gain*)—memengaruhi waktu respons, stabilitas momentum, dan tingkat keberhasilan robot dalam mengeksekusi lintasan potong (*intercept*) untuk mencetak gol.

2 Pemodelan Kinematika

Pada simulasi ini, pergerakan robot didasarkan pada model *differential drive*, di mana kecepatan linear dan angular robot secara utuh direpresentasikan melalui manipulasi kecepatan putar roda kanan dan kiri secara independen. Bab ini menguraikan persamaan matematis dan logika kontrol yang memandu perilaku Striker dan Goalkeeper.

2.1 Invers Kinematika Differential Drive

Untuk mengeksekusi perintah kecepatan linear (v_x) dan kecepatan sudut (ω_x) yang dihasilkan oleh sistem kontrol, robot harus menerjemahkannya menjadi kecepatan rotasi untuk masing-masing roda (ω_R dan ω_L). Berdasarkan parameter fisik berupa jari-jari roda ($r_w = 0,0975$ m) dan jarak antar roda ($L = 0,381$ m), persamaan invers kinematika yang digunakan pada kedua robot adalah:

$$\omega_R = \frac{v_x + \left(\frac{L}{2}\right)\omega_x}{r_w}$$

$$\omega_L = \frac{v_x - \left(\frac{L}{2}\right)\omega_x}{r_w}$$

Persamaan ini diaplikasikan langsung melalui fungsi `sim.setJointTargetVelocity` ke motor penggerak robot.

2.2 Sistem Kontrol Navigasi Striker

Robot `Robot_Pemain` menggunakan algoritma *Proportional Controller* untuk mengejar bola dan menggiringnya ke target koordinat absolut gawang ($\Delta x = x_{target} - x_{current}$).

1. **Kalkulasi Error Orientasi:** Sistem menghitung selisih sudut (*heading error*) antara arah robot saat ini ($\theta_{current}$) dengan posisi target menggunakan fungsi arctangent, yang dinormalisasi pada rentang $[-\pi, \pi]$:

$$\theta_e = \text{atan2}(\Delta y, \Delta x) - \theta_{current}$$

$$\theta_e = (\theta_e + \pi) \pmod{2\pi} - \pi$$

2. **Kontrol Kecepatan Sudut:** Rotasi robot merespons besaran error secara proporsional. Semakin besar deviasi sudut, semakin cepat robot berputar:

$$\omega_x = K_p \cdot \theta_e$$

di mana nilai *gain* proporsional (K_p) disetel melalui variabel `pemain_turn_gain`.

3. **Kontrol Kecepatan Linear Kondisional:** Untuk mencegah robot bergerak meleset dari bola saat berbelok tajam, kecepatan linear dinonaktifkan ($v_x = 0$) jika error sudut melebihi batas toleransi 45° ($\frac{\pi}{4}$ radian). Jika robot sudah menghadap target, kecepatan akan diatur ke `pemain_max_speed`.

2.3 Transformasi Koordinat Patroli Goalkeeper

Robot `Robot_Lawan_01` tidak mengejar bola, melainkan bergerak bolak-balik menuju *waypoint* imajiner yang dihasilkan di sekitar gawang. Untuk memastikan lintasan patroli selalu sejajar dengan mulut gawang terlepas dari orientasi gawang di dunia simulasi, program melakukan transformasi rotasi 2D.

Jika target relatif didefinisikan sebagai (x_{rel}, y_{rel}) dan orientasi gawang adalah θ_g , proyeksi koordinat target global (x_t, y_t) dihitung menggunakan matriks rotasi:

$$x_t = x_{gawang} + (x_{rel} \cos(\theta_g) - y_{rel} \sin(\theta_g))$$

$$y_t = y_{gawang} + (x_{rel} \sin(\theta_g) + y_{rel} \cos(\theta_g))$$

Setelah nilai (x_t, y_t) didapatkan, Goalkeeper menggunakan model kontrol kinematika yang sama dengan Striker untuk menuju titik tersebut, dan membalik arah patrolinya (mengubah tanda *amplitude*) apabila jaraknya terhadap target $\leq 0,2$ meter.

3 Implementasi Kode

Pengendalian robot diimplementasikan menggunakan bahasa Python yang terhubung ke simulator CoppeliaSim melalui ZeroMQ Remote API. Skrip ini secara simultan mengatur perilaku dua robot (Striker dan Goalkeeper) dalam sebuah *loop* utama yang dieksekusi secara *real-time*.

```

1 import time
2 import math
3 import numpy as np
4 from coppeliasim_zmqremoteapi_client import RemoteAPIClient
5
6 client = RemoteAPIClient()
7 sim = client.require('sim')
8
9 sim.startSimulation()
10
11 rw = 0.195 / 2
12 wheel_dist = 0.381
13
14 try:
15     p_pemain = sim.getObject("/Robot_Pemain")
16     p_pemain_RW = sim.getObject("/Robot_Pemain/rightMotor")
17     p_pemain_LW = sim.getObject("/Robot_Pemain/leftMotor")
18
19     p_lawan1 = sim.getObject("/Robot_Lawan_01")
20     p_lawan1_RW = sim.getObject("/Robot_Lawan_01/rightMotor")
21     p_lawan1_LW = sim.getObject("/Robot_Lawan_01/leftMotor")
22
23     bola_merah = sim.getObject("/Bola_Merah")
24 except Exception as e:
25     sim.stopSimulation()

```

```

26     exit()
27
28 # --- TUNABLE VARIABLES ---
29 # Striker (Robot_Pemain) tuning
30 pemain_max_speed = 0.35 # Adjust this to change forward speed (previously 0.6)
31 pemain_turn_gain = 2.0 # Adjust this to change how sharply the robot turns
32
33 # Goalkeeper (Robot_Lawan_01) tuning
34 patrol_amplitude = 0.8
35 moving_right = True
36
37 try:
38     while True:
39         pemain_pos = sim.getObjectPosition(p_pemain, sim.handle_world)
40         pemain_ori = sim.getObjectOrientation(p_pemain, sim.handle_world)
41
42         lawan1_pos = sim.getObjectPosition(p_lawan1, sim.handle_world)
43         lawan1_ori = sim.getObjectOrientation(p_lawan1, sim.handle_world)
44
45         bola_pos = sim.getObjectPosition(bola_merah, sim.handle_world)
46
47         # Exact center of the goal based on provided coordinates
48         gawang_x = 5.125
49         gawang_y = -0.075
50
51         # --- 1. ROBOT PEMAIN (STRIKER) LOGIC ---
52         # Target 1.0m deep inside the goal to ensure the pass crosses the line
53         target_g_x = gawang_x + 1.0
54         target_g_y = gawang_y
55
56         bg_vec = np.array([target_g_x - bola_pos[0], target_g_y - bola_pos[1]])
57         bg_dist = np.linalg.norm(bg_vec)
58         bg_dir = bg_vec / bg_dist if bg_dist > 0 else np.array([1.0, 0.0])
59
60         dist_pemain_bola = math.hypot(pemain_pos[0] - bola_pos[0], pemain_pos[1] -
bola_pos[1])
61
62         if dist_pemain_bola > 0.5:
63             target_p_x = bola_pos[0] - (bg_dir[0] * 0.4)
64             target_p_y = bola_pos[1] - (bg_dir[1] * 0.4)
65         else:
66             target_p_x = target_g_x
67             target_p_y = target_g_y
68
69         dx_p = target_p_x - pemain_pos[0]
70         dy_p = target_p_y - pemain_pos[1]
71         angle_p = math.atan2(dy_p, dx_p) - pemain_ori[2]
72         angle_p = (angle_p + math.pi) % (2 * math.pi) - math.pi
73
74         # Apply tunable variables here
75         wx_p = pemain_turn_gain * angle_p
76         vx_p = 0.0 if abs(angle_p) > math.pi / 4 else pemain_max_speed
77
78         wr_vel_p = (vx_p + (wheel_dist / 2) * wx_p) / rw
79         wl_vel_p = (vx_p - (wheel_dist / 2) * wx_p) / rw
80
81         sim.setJointTargetVelocity(p_pemain_RW, wr_vel_p)
82         sim.setJointTargetVelocity(p_pemain_LW, wl_vel_p)
83
84         # --- 2. ROBOT LAWAN 01 (GOALKEEPER) LOGIC ---
85         target_offset = patrol_amplitude if moving_right else -patrol_amplitude
86         # Patrol 0.3m in front of the goal line to block shots
87         target_l_x = gawang_x - 0.3
88         target_l_y = gawang_y + target_offset
89

```

```

90     dist_to_target = math.hypot(target_l_x - lawan1_pos[0], target_l_y -
    lawan1_pos[1])
91     if dist_to_target < 0.2:
92         moving_right = not moving_right
93
94     dx_l = target_l_x - lawan1_pos[0]
95     dy_l = target_l_y - lawan1_pos[1]
96     angle_l = math.atan2(dy_l, dx_l) - lawan1_ori[2]
97     angle_l = (angle_l + math.pi) % (2 * math.pi) - math.pi
98
99     wx_l = 2.0 * angle_l
100    vx_l = 0.0 if abs(angle_l) > math.pi / 4 else 0.3
101
102    wr_vel_l = (vx_l + (wheel_dist / 2) * wx_l) / rw
103    wl_vel_l = (vx_l - (wheel_dist / 2) * wx_l) / rw
104
105    sim.setJointTargetVelocity(p_lawan1_RW, wr_vel_l)
106    sim.setJointTargetVelocity(p_lawan1_LW, wl_vel_l)
107
108    time.sleep(0.01)
109
110 except KeyboardInterrupt:
111     pass
112 finally:
113     sim.setJointTargetVelocity(p_pemain_RW, 0.0)
114     sim.setJointTargetVelocity(p_pemain_LW, 0.0)
115     sim.setJointTargetVelocity(p_lawan1_RW, 0.0)
116     sim.setJointTargetVelocity(p_lawan1_LW, 0.0)
117     sim.stopSimulation()

```

Listing 1: Skrip Python Final Pengendalian Multi-Robot

3.1 Parameter Navigasi Global (Tunable Variables)

Untuk mempermudah proses penyetelan (*tuning*) responsivitas robot tanpa harus memodifikasi logika inti, beberapa variabel dideklarasikan di awal skrip:

- `pemain_max_speed = 0.35`: Membatasi kecepatan linear Striker agar pergerakan lebih stabil dan robot tidak melewati bola karena momentum yang berlebihan.
- `pemain_turn_gain = 2.0`: Konstanta penguat proporsional (K_p) yang mengatur seberapa tajam dan agresif Striker berbelok saat mengoreksi arah menuju bola.
- `patrol_amplitude = 0.8`: Jarak simpangan maksimum (kiri-kanan) Goalkeeper dari titik tengah gawang.

3.2 Logika Penentuan Target (Striker)

Berdasarkan koordinat absolut balok penyusun gawang, titik pusat **Gwang_Kuning** diketahui berada pada letak $x = 5,125$ dan $y = -0,075$. Agar Striker tidak hanya menghentikan bola di garis gawang, algoritma memproyeksikan target *waypoint* sejauh 1,0 meter ke dalam gawang:

$$x_{target} = 5,125 + 1,0 = 6,125$$

Logika pergerakan Striker dibagi menjadi dua tahap adaptif berdasarkan jaraknya terhadap bola (`dist_pemain_bola`):

1. **Fase Pendekatan (Jarak $\geq 0,5m$):** Robot akan menargetkan sebuah titik sejauh 0,4 meter di belakang bola, sejajar dengan garis lintasan menuju gawang. Hal ini memastikan robot sejajar dengan bola sebelum melakukan kontak.
2. **Fase Penggiringan (Jarak $\leq 0,5m$):** *Waypoint* robot bergeser langsung ke koordinat di dalam gawang (6,125, -0,075), secara efektif mendorong bola melewati garis gawang secara penuh.

3.3 Mekanisme Patroli Dinamis (Goalkeeper)

Goalkeeper mengeksekusi osilasi horizontal (*state machine* sederhana) untuk menutupi lebar gawang. Robot diberikan titik target yang berada 0,3 meter tepat di depan gawang ($x = 4,825$) untuk mencegah bola.

Nilai target sumbu- y selalu bergeser antara $\pm 0,8$ (ditentukan oleh `patrol_amplitude`). Apabila Goalkeeper telah mencapai radius jarak kurang dari 0,2 meter dari titik target saat ini, variabel boolean `moving_right` akan diinversi, sehingga robot secara otomatis berbalik arah untuk menutupi sisi gawang lainnya.

4 Hasil Simulasi

Eksperimen dilakukan dalam lingkungan CoppeliaSim dengan mengeksekusi skrip Python final yang mengatur pergerakan otonom `Robot_Pemain` (Striker) dan `Robot_Lawan_01` (Goalkeeper). Evaluasi difokuskan pada stabilitas lintasan, waktu respons, dan keberhasilan *interception* bola.

4.1 Analisis Lintasan Striker

Implementasi *Proportional Controller* pada Striker dengan parameter `pemain_max_speed = 0.35` dan `pemain_turn_gain = 2.0` menghasilkan manuver pergerakan yang mulus namun responsif. Terdapat tiga fase pergerakan yang teramati selama simulasi:

1. **Fase Koreksi Sudut Awal:** Pada detik-detik pertama, Striker yang mungkin tidak menghadap ke arah bola akan memprioritaskan putaran (*pivoting*) dengan $v_x \approx 0$ karena toleransi error sudut $\theta_e > \frac{\pi}{4}$. Hal ini mencegah robot bergerak menjauh dari lintasan ideal saat berbelok.
2. **Fase Pendekatan Terarah:** Setelah menghadap ke *waypoint* awal (0,4 meter di belakang bola), kecepatan memuncak di 0,35 m/s. Robot secara aktif mengoreksi *drift* kinematika, mempertahankan lintasan lurus menuju area *setup*.
3. **Fase Penetrasi Target:** Saat radius deteksi antara robot dan bola mencapai $< 0,5$ meter, *waypoint* dipindahkan seketika ke dalam gawang ($\Delta x = x_{gawang} + 1,0$). Transisi target ini memaksa robot untuk mendorong bola secara langsung dan konsisten melintasi garis gawang tanpa pengurangan kecepatan prematur.

Penggunaan target fiktif $x_{target} = 6,125$ terbukti sangat efektif. Pada iterasi awal (*unrecorded*), penempatan target tepat di tengah balok gawang seringkali menyebabkan bola berhenti memantul di garis (*dead center*), sehingga gagal memenuhi kriteria *goal*. Penambahan kedalaman target memastikan momentum gaya dorong tetap disalurkan secara maksimal hingga bola sepenuhnya masuk.

4.2 Evaluasi Manuver Patroli Goalkeeper

`Robot_Lawan_01` yang berfungsi sebagai Goalkeeper beroperasi menggunakan koordinat gawang absolut. Karena robot menggunakan kontrol *differential drive* untuk bergerak sepanjang sumbu- y , evaluasi mengacu pada kelancaran transisi arah:

- **Stabilitas Sumbu Samping:** Goalkeeper berhasil mempertahankan posisinya 0,3 meter sejajar di depan gawang, menyediakan area cakupan pertahanan yang konstan tanpa menyimpang ke belakang mendekati balok vertikal.
- **Transisi Waypoint:** Dengan menetapkan radius inversi arah pada 0,2 meter (`dist_to_target < 0.2`), robot dapat berbalik arah sebelum mencapai *overshoot* ekstrem akibat momentum rodanya. Skema pembalikan ini menghilangkan perilaku "bergetar" (*jitter*) yang kerap terjadi pada fungsi sinusoidal kontinu (seperti iterasi model patroli awal).

4.3 Kesimpulan Dinamika Sistem

Integrasi keseluruhan model navigasi menghasilkan simulasi dengan fidelitas interaksi yang tinggi:

1. Transformasi parameter global (titik x, y) menjadi instruksi lokal putaran roda roda (ω_R dan ω_L) secara *real-time* divalidasi dan berjalan dengan baik.
2. Kecepatan 0,35 m/s untuk Striker terbukti sebagai *sweet spot* untuk pengendalian bola. Kecepatan yang lebih tinggi seringkali menimbulkan efek *ricochet* karena sudut kontak yang tidak terkendali dengan Bola_Merah, sementara kecepatan yang lebih lambat rentan terhadap gangguan dari Goalkeeper.

3. Desain kontrol dengan arsitektur dua bagian independen—striker yang mengejar koordinat dinamis, berhadapan dengan target tetap yang dijaga secara statis—memberikan landasan yang representatif untuk pengembangan modul penghindaran rintangan dinamis (*dynamic obstacle avoidance*) pada penelitian lebih lanjut.

5 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan analisis yang telah dilakukan terhadap sistem kontrol multi-robot pada simulator CoppeliaSim, dapat ditarik beberapa kesimpulan utama:

1. **Efektivitas Kontrol Proporsional:** Penggunaan *Proportional Controller* dengan parameter $K_p = 2,0$ dan kecepatan linear maksimum 0,35 m/s terbukti efektif dalam menyeimbangkan responsivitas dan stabilitas. Striker mampu melakukan manuver koreksi arah dengan presisi tanpa mengalami osilasi berlebihan saat mendekati bola merah.
2. **Optimalisasi Target Penetrasi:** Strategi penempatan *waypoint* fiktif sejauh 1,0 meter di belakang garis gawang (pada $x = 6,125$) secara efektif mengatasi masalah bola berhenti prematur. Hal ini menegaskan bahwa dalam simulasi interaksi objek, target dorongan (*push target*) seringkali harus ditetapkan melebihi batas fisik akhir untuk menjamin perpindahan momentum yang konstan.
3. **Stabilitas Patroli Berbasis State:** Implementasi sistem pergerakan Goalkeeper yang memanfaatkan diskritisasi arah berdasarkan batas jarak target (*distance-based state switching*) menghasilkan lintasan osilasi yang lebih konsisten dibandingkan fungsi kontinu trigonometri (seperti sinusoida), sehingga menghilangkan anomali pergerakan (*jitter*) pada roda motor robot.
4. **Validasi Model Kinematika Differential Drive:** Penjabaran perintah kecepatan global pada ruang 2D (v, ω) menjadi variabel kecepatan putar aktuator individual (ω_R, ω_L) telah berhasil divalidasi dengan baik, ditunjukkan oleh kemampuan kedua agen robot untuk melacak koordinat spasial tanpa deviasi yang signifikan terhadap pemodelan teoritisnya.